A2 - REST API Backend

- Due Mar 23 by 11:59p.m.
- Points 120

In this assignment, you will build the backend REST API for a **temporary staffing platform** that connects dental clinics with qualified dental professionals for short-term work opportunities.

The goal of this assignment is to design and implement a backend system that supports time-sensitive job matching, enforces clear system states, and exposes well-defined REST APIs. This backend will serve as the foundation for Assignment 3, where you will build a React frontend that interacts with the APIs you design in this assignment.

Learning Outcomes

By completing this assignment, you will be able to:

- Implement a RESTful backend API that follows a given specification.
- Enforce application state and system invariants on the server side.
- Apply role-based access control to protect API endpoints and data.
- Use Prisma to define relational data models and manage database interactions.
- Implement backend logic that is robust to invalid input, unauthorized access, and conflicting user actions.

Overview

You will create a REST API that supports the following functionalities:

1. Use Types

The system supports three types of accounts:

- **Regular users**: dental professionals looking for temporary work.
- **Businesses**: dental clinics or offices posting short-term job opportunities.
- **Administrators (admins)**: staff members who manually verify businesses and user qualification; manage supported position types; and handle exceptional cases. For simplicity, all admins have the same privileges.

When the word "user" is used alone, it implies **either a regular user or a business.**

2. Registration and Authentication

- Regular users and businesses can register for accounts.
- Users must **activate** their account prior to their first log-in.
- Users can log in and log out of the system.
- Users can update their account information.
- Users can reset their password without being logged in.

3. Role-Based Access Control

- The system distinguishes between regular users, businesses, and administrators.
- Access to API endpoints is restricted based on the role of the authenticated user.
- Regular users and businesses may only access and modify resources they own or are authorized to view.

4. Administrative Management

- Administrators can **suspend** and unsuspend users.
- Administrators can **verify** business accounts, which allows them to create job postings.
- Administrators can create, update, and hide **position types**.
- Administrators can **approve** which position types a regular user is qualified to perform.

5. Position Types and Qualifications

- The platform supports a dynamic set of position types defined by administrators.
- Each job posting is associated with exactly one position type.
- Regular users may only participate in jobs for position types they have been approved for.
- Qualification details beyond position type matching are intentionally not modeled in the system.

6. Business Profiles and Job Posting

- Businesses can create and manage a business profile.
- Verified businesses can create job postings to advertise short-term staffing needs.
- Each job posting includes a position type, salary range, and work time window, i.e., proposed start and end time.
  - A posting automatically expires if there is not enough time left to complete a full negotiation window before the job starts.
  - A posting cannot have a start time that is too far into the future (a system-wide setting; defaults to 1 week).
- Business users can update or delete their own job postings, subject to constraints
  - A posting cannot be deleted or modified after the job has been filled.
    - Except to report the other party as no-show.
  - A posting cannot be deleted while involved in an active negotiation.
- Jobs that are completed, filled, canceled, or expired are not available for matching.

7. Availability and Discovery

- Regular users may indicate that they are open to work by setting their availability status to **available**.
- A regular user is considered **discoverable** by businesses (specifically, for their job postings) if all of the following conditions are met:
  - Their account is activated.
  - Their account is not suspended.
  - They are qualified for the job posting.
  - Their availability status is set to **available**.
  - They have been active within a system-defined inactivity window
  - They are not committed to another job whose working hours conflict with this job.
- A regular user's is automatically made available whenever they:
  - Explicitly set their availability status to **available**.
  - A regular user is automatically made available whenever they:
    - express interest in a job posting, or
    - have just ended a negotiation, regardless of outcome.
- Regular users who have been **inactive** for longer than the inactivity window (a system-wide setting: defaults to 5 minutes) are treated as **unavailable** and are not discoverable by businesses.
- Regular users can browse open job postings near them for which they are qualified for, regardless of their availability status.
- Search results include an estimated travel distance and travel time (ETA), computed using a simplified location model.
- They also include other fields such as salary range and creation time (of the job posting).
- Search results can be sorted, filtered, and must be paginated, i.e., broken into pages.

8. Expressing Interest

- Regular users may express interest in job postings they are qualified for, regardless of their availability status.
- Business users may express interest in available regular users who match a given position type.
- Expressing interest is a non-exclusive action and does not lock either party.
- When one party expresses interest, the other party is able to see and respond to that interest.
- A **negotiation** can be initiated by both parties when they mutually express interest AND neither parties are involved in an ongoing negotiation.
  - In the latter scenario, the party that initiated shall receive an error message accompanied with the estimated wait time.

9. Negotiation

- Each negotiation has a fixed exclusive negotiation window (a system-wide setting; defaults to 15 minutes).
- During this window:
  - neither party may initiate or receive other negotiation requests,
  - the associated job cannot be negotiated with other candidates.

- The system enforces that a user or job may participate in at most one active negotiation at a time.
- Either party may accept or reject a negotiation during the exclusive window.
- The business is allowed to modify the job posting during the negotiation, but doing so resets the negotiation state, i.e., any prior acceptance by either party is undone.
- If both parties accept, the job is marked as filled and the negotiation completes successfully.
- If either party rejects, the negotiation ends immediately.
- If the negotiation window expires without agreement, the negotiation ends automatically.
- When a negotiation ends without agreement, the regular user is automatically returned to the available state.

10. Filled Jobs and Commitments

- When a job becomes filled, the negotiation is completed successfully and the job is no longer available for browsing or matching.
- A filled job represents a confirmed work commitment between exactly one regular user and one business.
- The system does not allow a job to have more than one successful negotiation.
- A regular user who has a filled job is considered **committed** for the duration of the job's scheduled time window.
  - While committed, the regular user is not considered available during the **working hours** of the job, even if their available status is set.
    - In other words, if a different job posting has time conflict with a regular user's commitment (e.g., the job is 3pm to 9pm, and the regular user is committed to 12pm to 6pm), then that regular user should not appear in the candidate search result.
- Once the job's scheduled end time has passed, the commitment is considered complete.
- During the scheduled work period of a filled job (i.e., between the job's start time and end time), the business user may mark the regular user as a **no-show**.
- When a no-show is reported:
  - the job is canceled.
  - the regular user involved is immediately suspended by the system.

Limitations

To keep the assignment focused, the following features and behaviours are out of scope and do not need to be implemented:

- **Dispute handling and appeals:** the system does not provide automated dispute resolution, appeal workflows, evidence review, or strike systems. Handling disputes (including abuse of the no-show mechanism) is assumed to be performed manually by contacting the administrator, e.g., by phone.
- **Communication after a job is filled:** the system does not support communication between the regular user and business after a job has been filled. In particular, there is no post-job chat, follow-up

coordination, or scheduling communication.

- **Cancellation of filled job:** the system does not (currently) support canceling a commitment. We assume the parties involved will reach an agreement without involving the platform.
- **Record of negotiation**: the system does not record or guarantee delivery of chat messages exchanged during negotiation.
- **Payments and financial processing:** the system does not mediate, facilitate, or enforce payment between businesses and regular users. This includes payment processing, deposits, refunds, invoices, subscriptions, payroll, taxation, insurance, and any money transfer functionality.

Setup

Log in to **MarkUs (https://markus.teach.cs.toronto.edu/)** and go to CSC309. Then, navigate to the **A2** assignment. One group member should create the group using the "Create Group" button for A2, followed by inviting the other group member(s) to the group via the "Invite student(s)" button. You will need to know the utorids of your group members to complete the latter step. After that, click on the "Add Starter Files to Repository" to get the starter code.

For other group members, click on "Join Group" button once the invite has been sent. Confirm that you are joining the right group.

If you intend to work alone, click on the "Work alone" button.

Starter Code

The starter files for this assignment is in **A2** folder:

- **.gitignore**: Specifies files and folders to be ignored, ensuring that files you shouldn't commit aren't checked in.
- **src/app.js**: The source file for your Express application.
- **src/socket.js**: The source file for implementing real-time communication.
- **src/server.js**: The source file for the server that runs your application. *Do not change.*
- **package.json**: contains information about this package. *Do not change.*
  - You can run the `npm run clean` script to remove all generated files.
- **prisma/schema.prisma**: define your models here.
- **prisma/createsu.js**: a command line script that allows the system administrator to create the first administrator account.
- **prisma/seed.js:** a command line script that allows you to initialize your database (this will be run by the autotester during setup).

For this assignment, you may add any number of files that you want, as long as we can run `node src/server.js PORT` to start your backend server.

In addition to **express**, **prisma**, and **sqlite3**, the following packages are used for this assignment (they are already listed in package.json):

- **jsonwebtoken**: A library to generate and verify JSON Web Tokens (JWT) for authentication.
- **express-jwt**: Middleware for Express.js to validate JWTs and protect routes.
- **multer**: Middleware for handling multipart/form-data, primarily used for file uploads.
- **cors**: Middleware to enable Cross-Origin Resource Sharing (CORS) in Express.js applications.
- **uuid**: helps you create random uuids (for the reset token).
- **bcrypt**: (optional) allows you to encrypt user password.
- **socket.io**: enables real-time communication for negotiation chat during active negotiations.

You are not allowed to add, remove, or replace packages beyond this list. These are the only packages you may use for this assignment.

Creating an Administrator

An administrator account is required to create and manage other users. It cannot be created through API endpoints since those only create regular user or business accounts. Hence, we need a way to "seed" our database with an administrator.

Your first task is to complete a script named `createsu.js` in the `prisma` folder so that we can create administrators. The script should take 2 command line arguments: `email`, and `password` (in plaintext). Then, we should be able to create administrators with this command:

```
node prisma/createsu.js -- clive.su@mail.utoronto.ca SuperUser123!
```

Note the `--` before the email address (for those of you who already implemented `utorid` as the first argument, you may ignore this). Also, please ensure that the administrator is initialized as **activated**.

API Specifications

In this section, we describe the specifications of each API endpoint and the HTTP methods they support. Please adhere to the following guidelines when implementing the API.

Request Format

- For **GET** requests, parameters must be provided via the query string.
- For all other HTTP methods, the request payload must be sent in the request body as JSON, unless otherwise specified.
- All response bodies must be in JSON format.

Authentication and Authorization

- For each endpoint and HTTP method, a required role is specified. Possible roles are: **Any** (no authentication required), **Regular**, **Business**, and **Administrator**.
- If an endpoint requires authentication and the request is not authenticated, return **401 Unauthorized**.
- If an endpoint requires a specific role and the authenticated user does not have that role, return **403 Forbidden**.

Error Handling

- Errors must be returned using an appropriate HTTP status code and a JSON response of the form:

```
{ "error": "The error message" }
```

  - Some error responses may include additional properties if specified.
- The autotester will not check the error message contents; you may customize messages to help with your own debugging.

HTTP Status Codes

The autotester expects the following order of error checking:

1. Router-level errors
   - If no route matches the path, return **404 Not Found**.
   - If the path matches but the HTTP method is not supported, return **405 Method Not Allowed**.
2. Authentication
   - If the endpoint requires authentication and the request is not authenticated, return **401 Unauthorized**.
3. Role
   - If the request is authenticated but the account does not have the required role for the endpoint, return **403 Forbidden**.
4. Query String / Payload
   - If the request is invalid, e.g., missing required fields, extra fields not specified in the API, incorrect data types, values out of range, return **400 Bad Request**.
5. Endpoint-specific error
   - Any error specific to the endpoint, which may include 404 and 403.

You are encouraged to use middleware to unify and simplify this process.

/users

- **Method:** POST
- **Description:** Register a new regular user account
- **Role**: Any

- **Payload**:

Field	Required	Type	Description
first_name	Yes	string	
last_name	Yes	string	
email	Yes	string	Unique and valid format (do not check whether it actually exists)
password	Yes	string	8-20 characters, at least one uppercase, one lowercase, one number, one special character
phone_number	No	string	
postal_address	No	string	
birthday	No	string	ISO 8601 Format: "YYYY-MM-DD" (default: "1970-01-01")

- **Response**:
  - **201 Created** on success

```
{
  "id": 2,
  "first_name": "John",
  "last_name": "Doe",
  "email": "john.doe@mail.utoronto.ca",
  "activated": false,
  "role": "regular",
  "phone_number": "",
  "postal_address": "",
  "birthday": "1970-01-01",
  "createdAt": "2026-03-03T01:41:47.000Z",
  "resetToken": "ad71d4e1-8614-46aa-b96f-cb894e346506",
  "expiresAt": "2026-03-10T01:41:47.000Z"
}
```

  - **409 Conflict** if an account (regular user, business, or administrator) with that email address already exists.

Important: ids should be **unique** between regular users, businesses, and administrators, i.e., you should internally have an Account model to avoid duplicate authentication logic.

Upon account creation, a `resetToken` that expires in 7 days is returned, which can be used to activate the account (see `POST /auth/resets/:resetToken`). For this assignment, **you are not expected to send emails**.

/businesses

- **Method:** POST
- **Description:** Register a new business account
- **Role**: Any
- **Payload**:

Field	Required	Type	Description
business_name	Yes	string	Name of the business

Field	Required	Type	Description
owner_name	Yes	string	Primary business owner's full name
email	Yes	string	Unique and valid format (do not check whether it actually exists)
password	Yes	string	8-20 characters, at least one uppercase, one lowercase, one number, one special character
phone_number	Yes	string	
postal_address	Yes	string	
location	Yes	object	In the form of `{ "lon" : number, "lat" : number }` for longitude and latitude.

- **Response**:
  - **201 Created** on success

```
{
  "id": 3,
  "business_name": "Bahen Dental Clinic",
  "owner_name": "David Liu",
  "email": "david@bahendental.ca",
  "activated": false,
  "verified": false,
  "role": "business",
  "phone_number": "416-978-2011",
  "postal_address": "40 St George St, Toronto, ON M5S 2E4",
  "location": {
    "lon" : -79.3998729,
    "lat": 43.6598084
  },
  "createdAt": "2026-03-02T01:41:47.000Z",
  "resetToken": "ad71d4e1-8614-46aa-b96f-cb894e346506",
  "expiresAt": "2026-03-09T01:41:47.000Z"
}
```

  - **409 Conflict** if an account (regular user, business, or administrator) with that email address already exists.

/businesses

- **Method:** GET
- **Description:** Retrieve a list of businesses
- **Role**: Any, Admin
- **Payload**:

Field	Required	Type	Description
keyword	No	string	Search for keyword in business_name, email, postal_address, or phone_number (**Admin only:** owner_name)
activated	No	boolean	**Admin only:** filter by whether the account is activated
verified	No	boolean	**Admin only:** filter by verified status
sort	No	string	Choose which field to sort by. Options are `business_name` and `email` (**admin only**: `owner_name`). Do not sort by default.

Field	Required	Type	Description
order	No	string	Choose the sort order. Options are `asc` and `desc`. Defaults to `asc`.
page	No	number	Page number for pagination (default is 1)
limit	No	number	Number of objects per page (default is 10)

- **Response**:
  - **200 OK** : `count`, which stores the total number of results (after applying all filters), and `results`, which contains a list of business users.

```
{
  "count": 12,
  "results": [
    {
      "id": 3,
      "business_name": "Bahen Dental Clinic",
      "email": "david@bahendental.ca",
      "role": "business",
      "phone_number": "416-978-2011",
      "postal_address": "40 St George St, Toronto, ON M5S 2E4",
      /* These fields can only be seen by admins */
      "owner_name": "David Liu",
      "verified": false,
      "activated": true,
    },
    // More business objects...
  ]
}
```

  - **400 Bad Request:** If the user is not authenticated and one or more of the admin-only fields, e.g., `activated`, is specified.

/businesses/:businessId

- **Method:** GET
- **Description:** Retrieve a specific business
- **Role**: Any, Admin
- **Payload**: None
- **Response**:

```
{
  "id": 3,
  "business_name": "Bahen Dental Clinic",
  "email": "david@bahendental.ca",
  "role": "business",
  "phone_number": "416-978-2011",
  "postal_address": "40 St George St, Toronto, ON M5S 2E4",
  "location": {
    "lon" : -79.3998729,
    "lat": 43.6598084
  },
  "avatar": "/uploads/businesses/3/avatar.jpg",
  "biography": "Bahen Dental, opened in 2002, is a modern, patient-focused clinic committed to c
omfortable, personalized care.",
  /* These fields can only be seen by admins */
  "owner_name": "David Liu",
```

```
  "activated": true,
  "verified": false,
  "createdAt": "2026-03-02T01:41:47.000Z",
}
```

/auth/resets

- **Method:** POST
- **Description:** Request a password reset token that expires in 7 days.
- **Role**: Any
- **Payload**:

Field	Required	Type	Description
email	Yes	string	The email of an account holder (any of regular user, business, or administrator) who forgot their password

- **Response**
  - **202 Accepted** on success

    ```
    {
      "expiresAt": "2026-03-09T01:41:47.000Z",
      "resetToken": "ad71d4e1-8614-46aa-b96f-cb894e346506"
    }
    ```

  - **404 Not Found** if the email address does not exist.
  - **429 Too Many Requests** if another request is made from the same IP address within the reset cooldown (see `PATCH /system/reset-cooldown` for changing this value). Hint: your rate limiter can be implemented completely in memory. You may not use **express-rate-limit**, since we do not allow you to install additional packages (if you do, the autotester will break).

/auth/resets/:resetToken

- **Method:** POST
- **Description:** Given a reset token, activate an account OR reset its password.
- **Role**: Any
- **Payload**:

Field	Required	Type	Description
email	Yes	string	The email of the account holder who requested a password reset
password	No	string	8-20 characters, at least one uppercase, one lowercase, one number, one special character

- **Response**
  - **200 OK** on success
  - **401 Unauthorized** when the email address provided does not match the reset token.
  - **404 Not Found** if the reset token does not exist or has already been used.
  - **410 Gone** if the reset token expired.

If `password` is not specified, then the account's password is not changed. If the account is already activated and no password is specified, then the request has no effect, but the reset token is still considered "used". A subsequent request using the same `resetToken` will return **404 Not Found**.

/auth/tokens

- **Method:** POST
- **Description:** Authenticate an account holder and generate a JWT token
- **Role**: Any
- **Payload**:

Field	Required	Type	Description
email	Yes	string	The email address associated with an account
password	Yes	string	The password of the account holder

- **Response**
  - **200 OK**

    ```
    {
      "token": "jwt_token_here",
      "expiresAt": "2026-03-10T01:41:47.000Z"
    }
    ```

  - **401 Unauthorized** if the email and password combination are invalid.
  - **403 Forbidden** if the account is not activated.

Note that suspended accounts can still log-in, but they just can't make themselves available for discovery.

/users

- **Method:** GET
- **Description:** Retrieve a list of regular users
- **Role**: Admin
- **Payload**:

Field	Required	Type	Description
keyword	No	string	Search for keyword in first_name, last_name, email, postal_address, or phone_number
activated	No	boolean	Filter by whether the account is activated
suspended	No	boolean	Filter by suspended status
page	No	number	Page number for pagination (default is 1)
limit	No	number	Number of objects per page (default is 10)

- **Response**:

- `count`, which stores the total number of results (after applying all filters), and `results`, which contains a list of regular users.

```
{
  "count": 51,
  "results": [
    {
       "id": 2,
       "first_name": "John",
       "last_name": "Doe",
       "email": "john.doe@mail.utoronto.ca",
       "activated": true,
       "suspended": false,
       "role": "regular",
       "phone_number": "",
       "postal_address": "",
    },
    // More user objects...
  ]
}
```

/users/:userId/suspended

- **Method:** PATCH
- **Description:** Set the suspended status of a regular user
- **Role**: Admin
- **Payload**:

Field	Required	Type	Description
suspended	Yes	boolean	true or false

- **Response**:

```
{
  "id": 2,
  "first_name": "John",
  "last_name": "Doe",
  "email": "john.doe@mail.utoronto.ca",
  "activated": true,
  "suspended": true,
  "role": "regular",
  "phone_number": "",
  "postal_address": "",
}
```

/businesses/:businessId/verified

- **Method:** PATCH
- **Description:** Set the verified status of a business
- **Role**: Admin
- **Payload**:

Field	Required	Type	Description
verified	Yes	boolean	true or false

- **Response**:

```
{
  "id": 3,
  "business_name": "Bahen Dental Clinic",
  "owner_name": "David Liu",
  "email": "david@bahendental.ca",
  "activated": true,
  "verified": true,
  "role": "business",
  "phone_number": "416-978-2011",
  "postal_address": "40 St George St, Toronto, ON M5S 2E4",
}
```

/position-types

- **Method:** POST
- **Description:** Create a new position type
- **Role**: Admin
- **Payload**:

Field	Required	Type	Description
name	Yes	string	Name of the type of position
description	Yes	string	Description of the type of position
hidden	No	boolean	defaults to true

- **Response**:

```
{
  "id": 1,
  "name": "Orthodontist",
  "description": "a dentist who is qualified to treat irregularities in the teeth and jaws.",
  "hidden": true,
  "num_qualified": 0
}
```

  - `num_qualified`: the number of of qualified regular users for this position type.

/position-types

- **Method:** GET
- **Description:** Retrieve a list of position types
- **Role**: Regular, Business, Admin
- **Payload**:

Field	Required	Type	Description
keyword	No	string	Search for keyword in name or description
name	No	string	Sort by name (can be "asc" or "desc", defaults to "asc")
hidden	No	boolean	**Admin only:** Filter by whether the position type is hidden
num_qualified	No	string	**Admin only:** Sort by num_qualified (can be "asc" or "desc", defaults to "asc")
page	No	number	Page number for pagination (default is 1)

Field	Required	Type	Description
limit	No	number	Number of objects per page (default is 10)

- **Response**:
  - `count`, which stores the total number of results (after applying all filters), and `results`, which contains a list of position types. **Only admins are allowed to see hidden position types.**

```
{
  "count": 6,
  "results": [
    {
      "id": 1,
      "name": "Orthodontist",
      "description": "a dentist who is qualified to treat irregularities in the teeth and jaw
s.",
      /* These fields can only be seen by admins */
      "hidden": true,
      "num_qualified": 0
    }
    /* More position type objects... */
  ]
}
```

- For admins, their result should be sorted by `num_qualified` first, then by `name` on a tie.

/position-types/:positionTypeId

- **Method:** PATCH
- **Description:** Edit a position type's details
- **Role:** Admin
- **Payload:**

Field	Required	Type	Description
name	No	string	Name of the type of position
description	No	string	Description of the type of position
hidden	No	boolean	

- **Response**: Only the updated fields, so if `name` was updated, then:

```
{
  "id": 1,
  "name": "Orthodontist"
}
```

/position-types/:positionTypeId

- **Method:** DELETE
- **Description:** Delete a position type
- **Role:** Admin
- **Payload:** None
- **Response**:

- **204 No Content** on success.
- **409 Conflict** if the position has a non-zero number of qualified regular users.

/qualifications

- **Method:** GET
- **Description:** Retrieve a list of qualifications that need admin attention
- **Role:** Admin
- **Payload:**

Field	Required	Type	Description
keyword	No	string	Search for keyword in the associated user's first_name, last_name, email, or phone_number
page	No	number	Page number for pagination (default is 1)
limit	No	number	Number of objects per page (default is 10)

- **Response**:
  - `count`, which stores the total number of results, and `results`, which contains a list of qualifications.

```
{
  "count": 19,
  "results": [
    {
      "id": 69,
      "status": "submitted",
      "user": {
        "id": 7,
        "first_name": "Alice",
        "last_name": "Lin"
      },
      "position_type": {
        "id": 10,
        "name": "Dental Assistant (Level 1)",
      },
      "updatedAt": "2026-03-05T02:16:47.000Z"
    },
    /* More qualification objects... */
  ]
}
```

  - `status` should be one of "created", "submitted", "approved", "rejected", and "revised".
    - Only "submitted" and "revised" qualifications requires admin attention.

/qualifications

- **Method:** POST
- **Description:** Create a new qualification for a position type
- **Role:** Regular
- **Payload:**

Field	Required	Type	Description
position_type_id	Yes	number	The id of a visible position type (i.e., not hidden)
note	No	string	defaults to empty string

- **Response**:

```
{
  "id": 69,
  "status": "created",
  "note": "",
  "document": null,
  "user": {
    "id": 7,
    "first_name": "Alice",
    "last_name": "Lin"
  },
  "position_type": {
    "id": 10,
    "name": "Dental Assistant (Level 1)"
  },
  "updatedAt": "2026-03-02T02:16:47.000Z"
}
```

/qualifications/:qualificationId

- **Method:** GET
- **Description:** Retrieve a single qualification request
- **Role:** Admin, Regular, Business
- **Payload:** None
- **Response**:
  - **200 OK**

```
{
  "id": 69,
  "document": "/uploads/users/7/position_type/10/document.pdf",
  "note": "Attached is my Dental Assistant Level 1 certificate and supporting documentation f
or review.",
  "position_type": {
    "id": 10,
    "name": "Dental Assistant (Level 1)",
    "description": "Entry-level dental assistant."
  },
  "updatedAt": "2026-03-05T02:16:47.000Z",
  "user": {
    "id": 7,
    "first_name": "Alice",
    "last_name": "Lin",
    "role": "regular",
    "avatar": "/uploads/users/7/avatar.png",
    "resume": "/uploads/users/7/resume.pdf",
    "biography": "Alice Lin is a Toronto-based graduate student with an interest in healthcar
e technology and community outreach.",
    /* businesses cannot see these */
    "email": "alice.lin@mail.utoronto.ca",
    "phone_number": "416-555-3921",
    "postal_address": "123 College St, Toronto, ON M5T 1R5",
    "birthday": "1994-06-18",
    "activated": true,
    "suspended": false,
```

```
    "createdAt": "2026-01-02T14:27:45Z"
  },
  /* businesses cannot see this */
  "status": "submitted"
}
```

- **403 Forbidden** if any of the condition below is true:
  - a business attempts to view a non-approved qualification,
  - the qualification belongs to someone who did not express interest to any of the business's active job postings, or
  - the job posting that the person expressed interest in does not require this qualification (specifically, the position type of the qualification).
- **404 Not Found** if a regular user attempts to view a qualification that's not their own.

/qualifications/:qualificationId

- **Method:** PATCH
- **Description:** Update a qualification request
- **Role:** Admin, Regular
- **Payload:**

Field	Required	Type	Description
status	No	string	**Admin only**: can only update status from "submitted" or "revised" to "approved" or "rejected" **Regular only:** can only update status from "created" to "submitted"; or "approved" or "rejected" to "revised"
note	No	string	note regarding their qualification

- **Response**
  - **200 OK** (remember to update the updatedAt field)

```
{
  "id": 69,
  "status": "approved",
  "document": "/uploads/users/7/position_type/10/document.pdf",
  "note": "Attached is my Dental Assistant Level 1 certificate and supporting documentation f
or review.",
  "user": {
    "id": 7,
    "first_name": "Alice",
    "last_name": "Lin"
  },
  "position_type": {
    "id": 10,
    "name": "Dental Assistant (Level 1)"
  },
  "updatedAt": "2026-03-12T02:16:47.000Z"
}
```

  - **403 Forbidden** if the current authenticated user attempts to change `status` to a state not allowed by their role.

/system/reset-cooldown

- **Method:** PATCH
- **Description:** Change the reset cooldown for `/auth/resets`
- **Role:** Admin
- **Payload:**

Field	Required	Type	Description
reset_cooldown	Yes	number	the new reset cooldown (in seconds)

- **Response**:
  - **200 OK**

    ```
    {
      "reset_cooldown": 1
    }
    ```

  - **400 Bad Request** if the amount of time is less than zero.

Note: any system configuration related endpoints do not need to be persisted, i.e., they can simply be global variables.

/system/negotiation-window

- **Method:** PATCH
- **Description:** Change the duration of negotiation windows
- **Role:** Admin
- **Payload:**

Field	Required	Type	Description
negotiation_window	Yes	number	the new duration for a negotiation window (in seconds)

- **Response**:
  - **200 OK**

    ```
    {
      "negotiation_window": 600
    }
    ```

  - **400 Bad Request** if the duration is less than or equal to zero.

/system/job-start-window

- **Method:** PATCH
- **Description:** Change the limit for how far ahead you can set the job start time for a posting
- **Role:** Admin
- **Payload:**

Field	Required	Type	Description
job_start_window	Yes	number	the new amount of time between the current time and a new job's start time that is allowed (in hours)

- **Response**:
  - **200 OK**

    ```
    {
      "job_start_window": 24
    }
    ```

  - **400 Bad Request** if the amount of time is less than or equal to zero.

/system/availability-timeout

- **Method:** PATCH
- **Description:** Change the amount of time inactive before the regular user is considered unavailable
- **Role:** Admin
- **Payload:**

Field	Required	Type	Description
availability_timeout	Yes	number	the new amount of inactive time before user is considered unavailable (in seconds)

- **Response**:
  - **200 OK**

    ```
    {
      "availability_timeout": 60
    }
    ```

  - **400 Bad Request** if the amount of time is less than or equal to zero.

/users/me

- **Method:** GET
- **Description:** Retrieve the authenticated user's account profile.
- **Role:** Regular
- **Payload:** None
- **Response**:

```
{
  "id": 2,
  "first_name": "John",
  "last_name": "Doe",
  "email": "john.doe@mail.utoronto.ca",
  "activated": true,
  "suspended": false,
  "available": false,
  "role": "regular",
  "phone_number": "",
```

```
  "postal_address": "",
  "birthday": "1970-01-01",
  "createdAt": "2026-03-03T01:41:47.000Z",
  "avatar": null,
  "resume": null,
  "biography": "I am, or am I not?"
}
```

If a regular user has been inactive for longer than the availability timeout (see `PATCH /system/availability-timeout`), the server must report `available` to be `false`, regardless of whether the user previously set their availability to `true`, i.e., via `PATCH /user/me/available`). This affects what the API reports, not necessarily what is stored in the database, since `GET` requests must not modify objects.

/users/me

- **Method:** PATCH
- **Description:** Update fields on the authenticated regular user's account profile. Only the fields listed below may be updated.
- **Role:** Regular
- **Payload:**

Field	Required	Type	Description
first_name	No	string	new first name
last_name	No	string	new last name
phone_number	No	string	new phone number
postal_address	No	string	new postal address
birthday	No	string	new birthday in ISO 8601 format: `"YYYY-MM-DD"`
avatar	No	string	new avatar URL/path (or `null` to clear it)
biography	No	string	new biography

- **Response**:
  - **200 OK** returns an object containing only the fields that were updated.

```
{
  "id": 2,
  "phone_number": "416-555-0100",
  "postal_address": "40 St George St, Toronto, ON M5S 2E4"
}
```

For simplicity of the application, we currently do not support modifying the email address, since it is used for authentication purposes.

/users/me/available

- **Method:** PATCH
- **Description:** Change the availability status of the current authenticated user
- **Role:** Regular
- **Payload:**

Field	Required	Type	Description
available	Yes	boolean	true or false

- **Response**:
  - **200 OK**

    ```
    {
      "available": true
    }
    ```

  - **400 Bad Request** if the current user tries to set their availability status to true, but their account is suspended or they have have no approved qualifications.

/businesses/me

- **Method:** GET
- **Description:** Retrieve the authenticated business user's account profile.
- **Role:** Business
- **Payload:** None
- **Response**:

```
{
 "id": 3,
 "business_name": "Bahen Dental Clinic",
 "email": "david@bahendental.ca",
 "role": "business",
 "phone_number": "416-978-2011",
 "postal_address": "40 St George St, Toronto, ON M5S 2E4",
 "location": {
 "lon": -79.3998729,
 "lat": 43.6598084
 },
 "avatar": "/uploads/businesses/3/avatar.jpg",
 "biography": "Bahen Dental, opened in 2002, is a modern, patient-focused clinic committed to co
mfortable, personalized care.",
 "activated": true,
 "verified": false,
 "createdAt": "2026-03-02T01:41:47.000Z"
}
```

/businesses/me

- **Method:** PATCH
- **Description:** Update fields on the authenticated business user's account profile. Only the fields listed below may be updated.
- **Role:** Business
- **Payload:**

Field	Required	Type	Description

business_name	No	string	new business name
owner_name	No	string	new owner name
phone_number	No	string	new phone number
postal_address	No	string	new postal address
location	No	object	new location object containing `lon` (number) and `lat` (number)
avatar	No	string	new avatar URL/path (or `null` to clear it)
biography	No	string	new biography

- **Response**:
  - **200 OK** returns an object containing only the fields that were updated.

```
{
  "biography": "Now accepting evening appointments.",
  "location": {
    "lon": -79.4001,
    "lat": 43.6602
  }
}
```

For simplicity of the application, we currently do not support modifying the email address, since it is used for authentication purposes.

/users/me/avatar, /businesses/me/avatar, /users/me/resume, /qualifications/:qualificationId/document

- **Method:** PUT
- **Description:** Upload or replace a file
- **Role:** Regular, Business
- **Content-Type**: `multipart/form-data` (instead of `application/json`)
- **Payload:**

Field	Required	Type	Description
file	Yes	file	The file to upload. For images, must be an allowed image file type. For documents, only PDF is allowed.

- **Response**:
  - **200 OK**

```
{
  "avatar": "/uploads/users/2/avatar.png"
}
```

  - **400 Bad Request** if the request is not valid multipart form data, the file is missing, or the file type or size is invalid.

For this course, the allowed image types are `image/png` and `image/jpeg`; the only allowed document type is `application/pdf`.

/jobs

- **Method:** GET
- **Description:** Retrieve a paginated list of **open** job postings.
- **Role:** Regular
- **Payload:**

Field	Required	Type	Description
lat	No	number	Latitude of the authenticated user's current location
lon	No	number	Longitude of the authenticated user's current location
position_type_id	No	number	Filter by position type.
business_id	No	number	Filter by business.
sort	No	string	Choose which field to sort by. Options are `updatedAt`, `start_time`, `salary_min`, `salary_max`, `distance`, and `eta`. Default is `start_time`.
order	No	string	Choose the sort order. Options are `asc` and

			`desc`. Defaults to `asc`.
page	No	number	Page number for pagination (default is 1).
limit	No	number	Number of objects per page (default is 10).

- **Response**:
  - **200 OK**

```
{
  "count": 2,
  "results": [
    {
      "id": 101,
      "status": "open",
      "position_type": { "id": 10, "name": "Dental Assistant (Level 1)" },
      "business": { "id": 3, "business_name": "Bahen Dental Clinic" },
      "salary_min": 28,
      "salary_max": 35,
      "start_time": "2026-03-08T14:00:00.000Z",
      "end_time": "2026-03-08T22:00:00.000Z",
      "updatedAt": "2026-03-03T01:41:47.000Z",
      /* These two fields only appear if lat and lon are specified */
      "distance": 6,
      "eta": 12
    }
    /* More job objects... */
  ]
}
```

  - **400 Bad Request** sort by distance or eta is specified but lat and lon are not specified.
- **Note:** for distance calculation, please use the **Haversine formula (https://en.wikipedia.org/wiki/Haversine_formula)** with R = 6371.2 km. For ETA calculation, assume a constant speed of 30 km/hr.
- `distance` should be expressed in kilometers, and `eta` should be expressed in minutes.

/businesses/me/jobs

- **Method:** POST
- **Description:** Create a new job posting owned by the authenticated business.
- **Role:** Business
- **Payload:**

Field	Required	Type	Description

position_type_id	Yes	number	Position type for this posting.
salary_min	Yes	number	Minimum hourly rate (must be >= 0).
salary_max	Yes	number	Maximum hourly rate (must be >= `salary_min`).
start_time	Yes	string	Job start time in ISO 8601 format.
end_time	Yes	string	Job end time in ISO 8601 format (must be after `start_time`).
note	No	string	Optional note or description for the posting.

- **Constraints:** The job start time cannot be too far into the future (see `PATCH /system/job-start-window`), and the posting must leave enough time for a full negotiation window before `start_time` (otherwise the posting is considered expired).
- **Response**:
  - **201 Created**

```
{
  "id": 101,
  "status": "open",
  "position_type": { "id": 10, "name": "Dental Assistant (Level 1)" },
  "business": { "id": 3, "business_name": "Bahen Dental Clinic" },
  "worker": null,
  "note": "Come work for me",
  "salary_min": 28,
  "salary_max": 35,
  "start_time": "2026-03-08T14:00:00.000Z",
  "end_time": "2026-03-08T22:00:00.000Z",
  "updatedAt": "2026-03-03T01:41:47.000Z"
}
```

  - **400 Bad Request** if any required field is missing, has the wrong type, or violates timing constraints, e.g., either start_time or end_time is in the past.
  - **403 Forbidden** if the authenticated user is not allowed to create job postings (e.g., the business is not verified).

/businesses/me/jobs

- **Method:** GET
- **Description:** Get a paginated list of jobs created by the currently logged in business.
- **Role:** Business
- **Payload:**

Field	Required	Type	Description
position_type_id	No	number	Filter by position type.
salary_min	No	number	Filter by those with a greater minimum salary than this value.
salary_max	No	number	Filter by those with a greater maximum salary than this value.
start_time	No	string	Filter by jobs starting after at this time (ISO 8601 format).
end_time	No	string	Filter by jobs ending earlier than this time (ISO 8601 format).

status	No	array	Filter by one or more job statuses (default: `["open", "filled"]`). Possible statuses are "open", "expired", "filled", "canceled", and "completed".
page	No	number	Page number for pagination (default is 1).
limit	No	number	Number of objects per page (default is 10).

- **Response**:
  - **200 OK**

```
{
  "count": 2,
  "results": [
    {
      "id": 102,
      "status": "filled",
      "position_type": { "id": 10, "name": "Dental Assistant (Level 1)" },
      "business_id": 3,
      "worker": { "id": 12, "first_name": "Wayne", "last_name": "Gretzky" },
      "salary_min": 28,
      "salary_max": 35,
      "start_time": "2026-03-09T14:00:00.000Z",
      "end_time": "2026-03-09T22:00:00.000Z",
      "updatedAt": "2026-03-04T01:41:47.000Z"
    },
    /* More objects... */
  ]
}
```

/businesses/me/jobs/:jobId

- **Method:** PATCH
- **Description:** Edit an existing job.
- **Role:** Business
- **Payload:**

Field	Required	Type	Description
salary_min	No	number	Minimum hourly rate (must be >= 0).
salary_max	No	number	Maximum hourly rate (must be >= `salary_min`).
start_time	No	string	Job start time in ISO 8601 format.
end_time	No	string	Job end time in ISO 8601 format (must be after `start_time`).
note	No	string	Optional note or description for the posting.

- **Constraints:** Same as `POST /businesses/me/jobs`).
- **Response**:
  - **200 Ok** Returns the fields that were modified, plus `updatedAt`.

```
{
  "id": 101,
  "salary_min": 28,
  "salary_max": 35,
  "updatedAt": "2026-03-04T01:41:47.000Z"
}
```

  - **400 Bad Request** if the request violates timing constraints, e.g., either start_time or end_time is in the past.
  - **409 Conflict** if the job is no longer in the open state, or if the current time is past the start time (i.e., in the expired state)
- **Optional** (not tested): since updating a job posting may cause some interests to retroactively conflict with existing commitments, doing so should delete ALL existing interests associated with this job posting. However, an ongoing negotiation for this job, and its associated interest object should be unaffected.

/businesses/me/jobs/:jobId

- **Method:** DELETE
- **Description:** Delete an open or expired job posting
- **Role:** Business
- **Payload:** None
- **Response**:
  - **204 No Content** on success.
  - **409 Conflict** if the position is not in the "open" or "expired" state, or if there is an active negotiation.
- **Note**: Deleting a job posting should also delete any associated interests and negotiations.

/jobs/:jobId

- **Method:** GET
- **Description:** See the detail of a job
- **Role:** Regular, Business
- **Payload:**

Field	Required	Type	Description
lat	No	number	**Regular only:** Latitude of the authenticated user's current location
lon	No	number	**Regular only:** Longitude of the authenticated user's current location

- **Constraints:** Businesses should only see their own jobs, and regular users are only allowed to open jobs, OR the jobs they've filled, canceled (as in no-show), or completed.
- **Response**
  - **200 OK**

```
{
  "id": 101,
  "status": "open",
  "position_type": { "id": 10, "name": "Dental Assistant (Level 1)" },
  "business": { "id": 3, "business_name": "Bahen Dental Clinic" },
  "worker": null,
  "note": "Come work for me",
  "salary_min": 28,
  "salary_max": 35,
```

```
  "start_time": "2026-03-08T14:00:00.000Z",
  "end_time": "2026-03-08T22:00:00.000Z",
  "updatedAt": "2026-03-03T01:41:47.000Z",
  /* These two fields only appear if lat and lon are specified */
  "distance": 2.3,
  "eta": 12
}
```

- **400 Bad Request** if a business specifies either lat or lon (or both).
- **403 Forbidden** if the regular user does not qualify for the job.
- **404 Not Found** if `jobId` does not exist or the job does not belong to the authenticated business.

/jobs/:jobId/no-show

- **Method:** PATCH
- **Description:** Declare that the regular user did not show up to the job.
- **Role:** Business
- **Payload:** None
- **Response**:
  - **200 OK** (the associated regular to this job will become suspended as a result)

    ```
    {
      "id": 112,
      "status": "canceled",
      "updatedAt": "2026-04-01T01:41:47.000Z"
    }
    ```

  - **409 Conflict** if the job has not started yet (now < `start_time`), or is already over (now >= `end_time`), or is not filled yet.

/jobs/:jobId/interested

- **Method:** PATCH
- **Description:** Express (or withdraw) interest in a job posting.
- **Role:** Regular
- **Payload:**

Field	Required	Type	Description
interested	Yes	boolean	Set to `true` to express interest in the job, or `false` to withdraw interest.

- **Response**:
  - **200 OK**

    ```
    {
      "id": 500, /* this is an interest id */
      "job_id": 101,
      "candidate": {
        "id": 7,
        "interested": true
    ```

```
  },
  "business": {
    "id": 3
    "interested": null /* this means not yet decided */
  }
}
```

- **400 Bad Request** if `interested` is `false` but the user has not previously expressed interest in this job (i.e., there is nothing to withdraw).
- **403 Forbidden** if the regular user does not qualify for the job.
- **404 Not Found** if the job is deleted.
- **409 Conflict** if the job is no long available, e.g., filled, or if the user is currently in a negotiation for this job.

- **Note:** Successfully expressing interest in a job must reset the user's inactivity timer (internally).

/users/me/invitations

- **Method:** GET
- **Description:** Retrieve a list of job invitations for the authenticated regular user. Each invitation represents a job posting where a business has invited the user to express interest.
- **Role:** Regular
- **Payload:**

Field	Required	Type	Description
page	No	number	Page number for pagination (default is 1).
limit	No	number	Number of objects per page (default is 10).

- **Response**:
  - **200 OK** (Note that accepted invitations should not exist on this list)

```
{
  "count": 2,
  "results": [
    {
      "id": 101,
      "status": "open",
      "position_type": { "id": 10, "name": "Dental Assistant (Level 1)" },
      "business": { "id": 3, "business_name": "Bahen Dental Clinic" },
      "salary_min": 28,
      "salary_max": 35,
      "start_time": "2026-03-08T14:00:00.000Z",
      "end_time": "2026-03-08T22:00:00.000Z",
      "updatedAt": "2026-03-03T01:41:47.000Z"
    },
    {
      "id": 104,
      "status": "open",
      "position_type": { "id": 12, "name": "Dental Hygienist" },
      "business": { "id": 8, "business_name": "Spadina Smiles" },
      "salary_min": 40,
      "salary_max": 50,
      "start_time": "2026-03-10T13:00:00.000Z",
      "end_time": "2026-03-10T21:00:00.000Z",
      "updatedAt": "2026-03-04T19:20:00.000Z"
```

```
    }
  ]
}
```

/users/me/interests

- **Method:** GET
- **Description:** Retrieve a list of job postings that the authenticated regular user is currently interested in.
- **Role:** Regular
- **Payload:**

Field	Required	Type	Description
page	No	number	Page number for pagination (default is 1).
limit	No	number	Number of objects per page (default is 10).

- **Response**:
  - **200 OK**

```
{
  "count": 2,
  "results": [
    {
      "interest_id": 500,
      "mutual": true,
      "job": {
        "id": 101,
        "status": "open",
        "position_type": { "id": 10, "name": "Dental Assistant (Level 1)" },
        "business": { "id": 3, "business_name": "Bahen Dental Clinic" },
        "salary_min": 28,
        "salary_max": 35,
        "start_time": "2026-03-08T14:00:00.000Z",
        "end_time": "2026-03-08T22:00:00.000Z",
        "updatedAt": "2026-03-05T02:16:47.000Z"
      },
    },
    {
      "interest_id": 501,
      "mutual": false,
      "job": {
        "id": 105,
        "status": "open",
        "position_type": { "id": 12, "name": "Dental Hygienist" },
        "business": { "id": 9, "business_name": "CSSU Dental" },
        "salary_min": 67,
        "salary_max": 69,
        "start_time": "2026-03-06T13:00:00.000Z",
        "end_time": "2026-03-06T21:00:00.000Z",
        "updatedAt": "2026-03-05T03:12:10.000Z"
      },
    }
  ]
}
```

- **Notes: t**he `mutual` field indicates whether mutual interest has been reached. You can only initiate negotiation when `mutual` is true.

/jobs/:jobId/candidates

- **Method:** GET
- **Description:** List all *discoverable* qualified regular users for this job, and whether each has been invited by the business for this job.
- **Role:** Business
- **Payload:**

Field	Required	Type	Description
page	No	number	Page number for pagination (default is 1).
limit	No	number	Number of objects per page (default is 10).

- **Response**:
  - **200 OK**

```
{
  "count": 2,
  "results": [
    {
      "id": 7,
      "first_name": "Alice",
      "last_name": "Lin",
      "invited": true,
    },
    {
      "id": 12,
      "first_name": "Wayne",
      "last_name": "Gretzky",
      "invited": false,
    },
    /* More candidate objects... */
  ]
}
```

  - **404 Not Found** if this job posting does not belong to the authenticated business.
- **Note:** recall that if an available user is already committed to another job whose working hours conflicts with this job, then they are not considered discoverable.

/jobs/:jobId/candidates/:userId

- **Method:** GET
- **Description:** Retrieve summary details about a regular user and their qualification for a given job.
- **Role:** Business
- **Payload:** None
- **Response**:
  - **200 OK**

```
{
  "user": {
    "id": 7,
    "first_name": "Alice",
    "last_name": "Lin",
```

```
    "avatar": "/uploads/users/7/avatar.png",
    "resume": "/uploads/users/7/resume.pdf",
    "biography": "Alice Lin is a Toronto-based graduate student with an interest in healthcar
e technology and community outreach.",
    "qualification": {
      "id": 69,
      "position_type_id": 10,
      "document": "/uploads/users/7/position_type/10/document.pdf",
      "note": "Attached is my Dental Assistant Level 1 certificate and supporting documentati
on for review.",
      "updatedAt": "2026-03-05T02:16:47.000Z"
    },
    /* These fields should only be displayed if the candidate filled this job */
    "email": "alice.lin@mail.utoronto.ca",
    "phone_number": "416-555-3921"
  },
  "job": {
    "id": 101,
    "status": "open",
    "position_type": {
      "id": 10,
      "name": "Dental Assistant (Level 1)",
      "description": "Entry-level dental assistant."
    },
    "start_time": "2026-03-08T14:00:00.000Z",
    "end_time": "2026-03-08T22:00:00.000Z"
  }
}
```

    - The returned qualification object is the regular user's qualification for the job's position type.
  - **403 Forbidden** if the regular user is no longer discoverable.
    - Exception: the candidate who filled this job should be unconditionally accessible until the job's end time, even if they're no longer discoverable.
  - **404 Not Found** if this job posting does not belong to the authenticated business, if `jobId` does not exist, or if `userId` does not exist.

/jobs/:jobId/candidates/:userId/interested

- **Method:** PATCH
- **Description:** Invite (or withdraw an invitation for) a regular user to express interest in this job posting.
- **Role:** Business
- **Payload:**

Field	Required	Type	Description
interested	Yes	boolean	Set to true to invite the user for this job, or false to withdraw the invitation.

- **Response**:
  - **200 OK**

```
{
  "id": 500, /* this is an interest id */
  "job_id": 101,
  "candidate": {
    "id": 7,
    "interested": true
  },
```

```
  "business": {
    "id": 3
    "interested": true
  }
}
```

- **400 Bad Request** if `interested` is false but the business has not previously invited this user for this job (i.e., there is nothing to withdraw).
- **403 Forbidden** if the user is no longer discoverable.
- **404 Not Found** if this job posting does not belong to the authenticated business, if `jobId` does not exist, or if `userId` does not exist.
- **409 Conflict** if the job is not in a state where the business is allowed to express interest in a candidate (job is not open).

/jobs/:jobId/interests

- **Method:** GET
- **Description:** Retrieve a list of regular users who are currently interested in this job posting.
- **Role:** Business
- **Payload:**

Field	Required	Type	Description
page	No	number	Page number for pagination (default is 1).
limit	No	number	Number of objects per page (default is 10).

- **Response**:
  - **200 OK**

```
{
  "count": 2,
  "results": [
    {
      "interest_id": 500,
      "mutual": true,
      "user": {
        "id": 7,
        "first_name": "Alice",
        "last_name": "Lin",
      }
    },
    {
      "interest_id": 505,
      "mutual": false,
      "user": {
        "id": 12,
        "first_name": "Wayne",
        "last_name": "Gretzky",
      }
    }
    /* More objects... */
  ]
}
```

  - **404 Not Found** if this job posting does not belong to the authenticated business.

- **Note:** `mutual` is true if both parties have expressed interests, which is the precondition for initiating negotiation.

/negotiations

- **Method:** POST
- **Description:** Start a negotiation for a job.
- **Role:** Regular, Business
- **Payload:**

Field	Required	Type	Description
interest_id	Yes	number	The ID of an existing interest object

- **Response**:
  - **201 Created**

```
{
  "id": 900,
  "status": "active",
  "createdAt": "2026-03-05T02:18:00.000Z",
  "updatedAt": "2026-03-05T02:18:00.000Z",
  "expiresAt": "2026-03-05T02:33:00.000Z",
  "job": {
    "id": 101,
    "status": "open",
    "position_type": { "id": 10, "name": "Dental Assistant (Level 1)" },
    "business": { "id": 3, "business_name": "Bahen Dental Clinic" },
    "salary_min": 28,
    "salary_max": 35,
    "start_time": "2026-03-08T14:00:00.000Z",
    "end_time": "2026-03-08T22:00:00.000Z"
  },
  "user": {
    "id": 7,
    "first_name": "Alice",
    "last_name": "Lin"
  },
  "decisions": { "candidate": null, "business": null }
}
```

  - **403 Forbidden** if:
    - the interest is not mutual (i.e., both parties have not expressed interest), or
    - the regular user is not currently discoverable for this job posting.
  - **404 Not Found** if the interest does not exist, or if the authenticated user is neither the business nor the candidate.
  - **409 Conflict** if a negotiation cannot be started because of either of these situations:
    - either party is already in an active negotiation, or
    - the job is no longer in a state where negotiation can be started (e.g., filled, canceled, completed, expired).
- **Notes:**

- Creating a negotiation is a no-op if an active negotiation already exists for the same `interest_id`; in that case, the server should return the existing negotiation with **200 OK** instead of creating a new one.
- The negotiation expiry time is based on a system-defined negotiation window (see `PATCH /system/negotiation-window`)

/negotiations/me

- **Method:** GET
- **Description:** Retrieve the authenticated users' current negotiation.
- **Role:** Regular, Business
- **Payload:** None
- **Response**:
  - **200 OK**

```
{
  "id": 900,
  "status": "active",
  "createdAt": "2026-03-05T02:18:00.000Z",
  "expiresAt": "2026-03-05T02:33:00.000Z",
  "updatedAt": "2026-03-05T02:18:00.000Z",
  "job": {
    "id": 101,
    "status": "open",
    "position_type": { "id": 10, "name": "Dental Assistant (Level 1)" },
    "business": { "id": 3, "business_name": "Bahen Dental Clinic" },
    "salary_min": 28,
    "salary_max": 35,
    "start_time": "2026-03-08T14:00:00.000Z",
    "end_time": "2026-03-08T22:00:00.000Z",
    "updatedAt": "2026-03-05T02:19:55.000Z"
  },
  "user": {
    "id": 7,
    "first_name": "Alice",
    "last_name": "Lin"
  },
  "decisions": {
    "candidate": null,
    "business": null
  }
}
```

  - **404 Not Found** if the authenticated user is not involved in an active negotiation.

/negotiations/me/decision

- **Method:** PATCH
- **Description:** Set the authenticated party's decision for an active negotiation.
- **Role:** Regular, Business
- **Payload:**

Field	Required	Type	Description
decision	Yes	string	The decision for the authenticated party. Must be one of "accept" or "decline".
negotiation_id	Yes	number	The negotiation ID the authenticated user is expecting to make a decision for

- **Response**:
  - **200 OK**

```
{
  "id": 900,
  "status": "active",
  "createdAt": "2026-03-05T02:18:00.000Z",
  "expiresAt": "2026-03-05T02:33:00.000Z",
  "updatedAt": "2026-03-05T02:20:10.000Z",
  "decisions": {
    "candidate": "accept",
    "business": null
  }
}
```

  - **404 Not Found** if the authenticated user is not involved in an active negotiation.
  - **409 Conflict** if `negotiation_id` does not match the authenticated user's current active negotiation, or if the negotiation is not in a state where decisions can be made.
- **Notes:**
  - Both parties must accept in order for the negotiation to become successful (status = "success")
    - A successful negotiation will update the job posting's status to "filled".
  - Only one party needs to decline in order for the negotiation to fail (status = "failed")
  - If a job posting is updated during the negotiation, both party's decisions must be reset to `null`.
  - In the case of an expired negotiation, the authenticated user should receive an error since it's no longer active.
  - When a negotiation ends, the regular user's inactivity timer should be reset, regardless of outcome.
  - When a negotiation fails, the interest for both the business and the user should be reset / nulled.

Real-time Communication

This project uses Socket.IO for (1) real-time negotiation chat and (2) notifications when a new negotiation starts. You are free to implement additional notifications, but only the events listed below are required for this assignment.

Rooms

- Each negotiation has a dedicated room named: `negotiation:<negotiationId>`
- Each authenticated user has a dedicated room named: `account:<accountId>`, where `accountId` is the id for a business or a regular user.

Authentication

- The client should connect to the server, e.g., localhost:3000 (same port as the regular server) with the following payload: `{ auth: { token: "<jwt token>" } }).`
  - On failure, disconnect the socket immediately.
  - On success, associate the socket with the authenticated user, e.g., by setting `socket.userId`. The server should then automatically join the socket to `account:<accountId>` room.

Notification: New Negotiation

- **From:** Server
- **Event:** `negotiation:started`
- **To:** `account:<accountId>` room for each party in the negotiation
- **Description:** Notifies a user that a negotiation has started.
- **Payload**

```
{
  "negotiation_id": 900,
}
```

- **Note:** the server should join each party to negotiation:<negotiationId> when a negotiation starts.

Chat Messages

- **From:** Regular, Business
- **Event:** `negotiation:message`
- **Description:** Send a chat message to the other party in the current negotiation.
- **Payload**:

```
{
  "negotiation_id": 900,
  "text": "Hi! I can confirm I'm available. Would you like me to arrive 10 minutes early?"
}
```

- **Constraints:**
  - The sender must be authenticated
  - the authenticated user is a party in `negotiation_id` and that the negotiation is currently active.
- **Server Response**:

```
{
  "negotiation_id": 900,
  "sender": {
    "role": "business",
    "id": 3
  },
  "text": "Hi! That works for us. Please arrive 10 minutes early.",
```

```
  "createdAt": "2026-03-05T02:21:10.000Z"
}
```

- **To:** `negotiation:<negotiationId>` room
- **Error Handling**: upon any of the following errors, emit an event named "negotiation:error" with the payload `{ "error": "one of the 4 below", "message": "whatever you want" }`
  - *Not authenticated*
  - *Not part of this negotiation*
  - *Negotiation not found* (or not active)
  - *Negotiation mismatch* (negotiation id is not that of the authenticated user's active negotiation)

Getting Started

This is a challenging assignment. We strongly recommend that you start early and work incrementally. The system has many interacting components, and attempting to implement everything at once will make debugging significantly harder.

1. Understand the Requirements

Before writing any code, carefully read the entire handout and make sure you understand:

- The expected system behaviour and constraints.
- The user stories, which define how the system is expected to behave from different roles' perspectives.
- The API specifications, including request formats, response formats, and required status codes.
- The business rules and invariants that must be enforced by the backend.

Spending time up front to understand the rules and edge cases will save substantial time later.

2. Plan Your Database Schema

Start by identifying the core entities and their relationships. Draft your schema in `schema.prisma` before implementing endpoints. Drawing a diagram (e.g., using dbdiagram.io) is strongly recommended. Expect to iterate on your schema as you uncover new requirements.

4. Incremental Development

Avoid trying to implement the entire system at once. Start with foundational pieces, then layer in more complex behaviour. Implement and test endpoints one at a time. Start with simpler CRUD-style endpoints before tackling logic-heavy workflows.

5. Testing Your System

Test continuously as you implement features. Do not wait until the end of the assignment to test.

- Use Postman or similar tools to test each endpoint as soon as it is implemented.
- Test both valid requests (happy paths) and error cases (invalid input, missing authentication, insufficient role, conflicting state).
- Inspect the database after major changes to ensure that state transitions are correct and consistent.

As the system becomes more complete, test **end-to-end workflows** that involve multiple roles and multiple endpoints:

- Simulate realistic scenarios such as job discovery, expressing interest, negotiation, and job filling.
- Verify that endpoints interact correctly and that system invariants are preserved across requests.
- Cross-check your implementation against the user stories to ensure all required behaviour is covered.

Do not rely solely on the autotester to discover logic errors. Many issues only surface when exercising full workflows rather than individual endpoints.

Suggestions

The following guidance addresses common sources of bugs and integration failures. Ignoring these often leads to systems that work locally but fail under full testing.

1. Derived State

The assignment involves many derived states, such as discoverability, inactivity (which affects availability), and expiration (of jobs and negotiations). Do not store derived state persistently in the database. Doing so easily leads to inconsistencies and hard-to-debug errors. Instead, store raw facts such as timestamps and flags, and derive the actual state when needed. For example, you should implement a single utility function that determines whether a regular user is discoverable, and use it everywhere. Do not duplicate discoverability checks across endpoints.

2. Lazy Cleanup

Based on the above, do not rely on `setTimeout`, `setInterval`, or cron-style jobs to enforce correctness of timeouts. Timers are not exact, are unreliable across server restarts, and cannot be assumed to run before the next request that needs the information. Even if timers are used, correctness must still be enforced at request time.

Instead, on each relevant request, compare the current time with stored timestamps and treat expired entities as expired immediately, even if no cleanup has occurred yet. For example, a regular user's `available` field may still be `true` after an inactivity timeout, but should be treated as unavailable when handling requests. Cleanup may be performed later when convenient (e.g., during a subsequent non-idempotent request). Recall that GET requests are idempotent and should not modify the database.

3. One "now" per Request

Calling `Date.now()` multiple times during a single request can produce subtle boundary bugs, so you should compute the current time once at the beginning of a request and reuse it throughout.

4. Atomicity

The rule that a user or business may participate in at most one active negotiation is a common source of race conditions. For example, both parties may simultaneously attempt to create a negotiation, resulting in multiple active negotiations. You must ensure this invariant is enforced using atomic operations or transactions. Do not rely on separate “check then create” logic without mutual exclusion.

5. Choice of ID

You are encouraged to use autoincrementing integers as IDs for models, but this is not required. The autotester will use whatever IDs your API returns.

Final Notes

- Start early. This assignment rewards iteration and careful reasoning.
- Ask for help when stuck. Piazza, office hours, and tutorials exist for a reason.
- Design for correctness first, then refactor for clarity.

Good luck, and happy coding.

Submission

Once you have tested your code and verified that it works according to specification, **and committed it locally** (check that by running `git status`), you can `git push` it back to MarkUs. We will collect and grade the last version pushed to MarkUs after the assignment deadline. Note that whatever you get from the autotester will be your final mark, therefore, you are recommended to run it before your final submission.

NOTE: the autotester is scheduled to be released after reading week.

Academic Integrity

Assignments are **individual work** which means that you will not be allowed to seek advice from other students or copy/paste someone else's code, even open-source codes from the internet. However, you are allowed to look at online resources, tutorials, and Q&A websites to solve the problems. The entire code must be written by yourself.